

Data Analysis Programming

Semester 2026-I

Workshop No. 4 — Interactive Dashboards with Plotly Dash

Eng. Carlos Andrés Sierra, M.Sc.

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

Welcome to the fourth workshop of the *Data Analysis Programming* course!

In **Workshop #1**, you defined your project and validated your data sources. In **Workshop #2**, you built the Bronze and Silver pipeline layers using Apache Airflow and Docker. In **Workshop #3**, you implemented the Gold layer using PySpark, defined governance KPIs, and structured the storytelling aggregations. Now, you will bring everything together into the user-facing layer: two interactive web dashboards built with **Plotly Dash**, running locally and powered exclusively by the Gold layer Parquet files.

This workshop is focused on **quality and integration**. The pipeline is now in place; your mission is to ensure all layers are correctly connected and to deliver two polished, functional dashboards that genuinely communicate insight to your defined functional user.

General Workshop Definition: Interactive dashboards are the final interface between a data pipeline and its human audience. In this workshop, teams will build two separate Plotly Dash web applications — one for data governance and one for functional user storytelling — both reading data exclusively from the Gold Parquet files. The focus is on visualization quality, layout clarity, and narrative coherence, ensuring that each dashboard fulfills the user stories defined in Workshop #1 and communicates meaningful insight about public opinion on the chosen topic.

Carlos Andrés Sierra, Computer Engineer, M.Sc. in Computer Engineering, Full-time Adjunct Professor at Universidad Distrital Francisco José de Caldas.

Any comment or concern about this document can be sent to Carlos A. Sierra at: *cavir-guezs@udistrital.edu.co*.

Workshop Scope and Objectives

- **End-to-End Pipeline Integration:** Verify and ensure that the full Medallion pipeline (Bronze → Silver → Gold) runs correctly end-to-end within the Docker environment, with all layers producing consistent and traceable outputs.
- **Governance Dashboard:** Build a Plotly Dash web application that reads the governance Gold Parquet file and visualizes all defined KPIs through clear, professional charts and metric cards accessible to a technical audience.
- **Storytelling Dashboard:** Build a second Plotly Dash web application that reads the storytelling Gold Parquet file and presents sentiment analysis results, keyword insights, and temporal trends through an interactive, narrative-driven interface designed for the functional user.
- **Visualization Quality:** Ensure all charts are clearly labeled, properly scaled, color-coherent, and supported by descriptive titles and annotations that guide the user through the insights.
- **Integration Adjustments:** Apply any final corrections to the pipeline or Gold summaries needed to ensure the dashboards display accurate, complete, and meaningful data.

Dashboard Development Methodology

1. End-to-End Integration Verification:

- Run the complete pipeline end-to-end from a clean state: Bronze ingestion DAG → FileSensor Silver DAG → Gold PySpark DAG
- Verify that the Gold Parquet files (`governance*.parquet` and `storytelling*.parquet`) contain complete, accurate, and up-to-date data
- Identify and resolve any integration issues: schema mismatches between layers, missing fields in Gold summaries, incorrect aggregations, or broken DAG dependencies
- Document all integration adjustments made in this workshop, distinguishing them from the improvements already covered in Workshop #3
- Ensure that both dashboards can be started independently and read their respective Gold Parquet files without errors

2. Governance Dashboard — Design and Implementation:

- Design the layout of the governance dashboard before coding: define which KPI cards, charts, and tables will be included and their arrangement
- Implement the Plotly Dash application in `dashboard/governance_app.py` with the following minimum components:

- *KPI Summary Cards*: Numeric metric cards displaying key governance figures (total records, null rates, duplicate rates, schema compliance rate)
- *Null Rate Bar Chart*: Horizontal bar chart showing null rate per field, ordered by severity
- *Volume Over Time Chart*: Line or bar chart showing ingested record volume per time period and per source
- *Outlier Rate Chart*: Visualization of outlier rates per numeric field
- *Data Quality Summary Table*: Tabular view of all governance KPIs with current values and defined thresholds
- Use Pandas to load the governance Gold Parquet file (`pd.read_parquet()`) and pass data to Dash components
- Apply a consistent color scheme and professional layout using Dash Bootstrap Components or custom CSS
- Configure the application to run at `localhost:8050`
- Ensure the dashboard auto-refreshes or provides a manual refresh mechanism to reflect the latest Gold Parquet file

3. Storytelling Dashboard — Design and Implementation:

- Design the dashboard narrative layout: define the story flow from overview to detail, guiding the functional user through the key insights
- Implement the Plotly Dash application in `dashboard/storytelling_app.py` with the following minimum components:
 - *Sentiment Distribution Pie/Donut Chart*: Overall breakdown of positive, negative, and neutral records
 - *Sentiment Trend Line Chart*: Average sentiment score over time (daily or weekly), revealing opinion shifts
 - *Top Keywords Bar Chart*: Horizontal bar chart of the most frequent terms in the corpus, colored by associated sentiment
 - *Source Comparison Chart*: Side-by-side sentiment and volume comparison between the API source and the scraping source
 - *Volume Activity Chart*: Record count per time period to highlight peaks of public discussion activity
 - *Narrative Summary Card*: A text component displaying the key insight or conclusion derived from the data, written for the functional user
- Use Pandas to load the storytelling Gold Parquet file and pass data to Dash components
- Design the interface for the functional user profile defined in Workshop #1: use plain language labels, avoid technical jargon, and prioritize insight over raw numbers
- Configure the application to run at `localhost:8051`

- Ensure all chart interactions (hover, zoom, filter) are functional and add value to the user experience

4. Visualization Quality Standards:

- Every chart must include: a descriptive title, labeled axes (with units where applicable), a legend (where applicable), and a source annotation
- Use a consistent color palette across both dashboards that is accessible (colorblind-friendly) and visually coherent
- Sentiment-related charts must use a consistent color encoding: green for positive, red for negative, grey or yellow for neutral
- Avoid chart types that obscure data: prefer bar charts over pie charts for comparisons with more than three categories; use line charts for time series
- Ensure dashboards are legible at standard screen resolutions (1080p) and do not require horizontal scrolling
- Include a header section in each dashboard with the project title, topic angle, team name, and data last-updated timestamp

5. Final Integration and Reproducibility:

- Ensure the entire project can be reproduced from a fresh clone of the repository following only the instructions in the `README.md`
- Verify that `docker-compose up` correctly starts all Airflow services and that both dashboard applications can be started independently with a single command
- Confirm that all DAGs are visible and triggerable from the Airflow UI at `localhost:8080`
- Confirm that the governance dashboard is accessible at `localhost:8050` and the storytelling dashboard at `localhost:8051`
- Perform a final end-to-end test: trigger all DAGs in sequence and verify that both dashboards update correctly with the new data

Deliverables and Documentation

1. Dashboard Implementation Report (PDF):

- *Executive Summary*: Overview of both dashboards, integration status, and key visualization decisions
- *Integration Adjustments*: Description of any pipeline or Gold layer corrections made in this workshop
- *Governance Dashboard Documentation*: Layout description, component list, KPI mapping, and design rationale
- *Storytelling Dashboard Documentation*: Layout description, component list, user story mapping, and narrative rationale

- *Visualization Design Decisions:* Justification for chart type selections, color palette, and layout choices
- *Reproducibility Evidence:* Screenshots demonstrating the full pipeline running in Airflow UI and both dashboards accessible in a browser
- *Challenges and Resolutions:* Discussion of integration or visualization challenges encountered and how they were resolved

2. Implementation Code:

- Governance dashboard application (`dashboard/governance_app.py`)
- Storytelling dashboard application (`dashboard/storytelling_app.py`)
- Any shared utility modules (data loading, color palette definitions, layout components)
- Updated `requirements.txt` including `dash`, `plotly`, `dash-bootstrap-components`, and `pandas`
- Any additional Docker or startup configuration for running the dashboard applications

3. Visual Documentation:

- Full-page screenshots of both dashboards showing all components populated with real Gold data
- Screenshots of the Airflow UI showing all three DAGs (Bronze, Silver, Gold) with successful run history
- A short screen-recorded or narrated walkthrough of both dashboards (optional but strongly recommended)

4. Final Repository State:

- Add a `workshop_4/` folder containing the report PDF and dashboard screenshots
- Update the repository `README.md` with complete setup and execution instructions: how to start Docker, trigger DAGs, and launch both dashboards
- Ensure the repository structure is clean, well-organized, and navigable for an external reviewer
- Tag the repository commit corresponding to this workshop delivery with `v1.0-workshop4`

Submission Requirements

- Submit your complete Dashboard Implementation Report as a single PDF through the designated course platform
- Include your GitHub repository link with all dashboard code, updated pipeline code, and full screenshot documentation
- All documentation must be in **English** and follow professional technical writing standards
- Ensure proper citations for Plotly Dash documentation, visualization best practices, and any referenced design frameworks
- Both dashboards must be functional and display real data from the Gold layer — screenshots in the report must reflect actual pipeline outputs, not placeholder data

Important Guidelines and Considerations

- **Quality over Quantity:** This workshop rewards dashboard quality, not the number of charts. A dashboard with five well-designed, clearly labeled, and meaningfully interpreted charts is far more valuable than one with fifteen confusing or redundant visualizations.
- **Gold as the Only Data Source:** Both dashboards must read exclusively from Gold Parquet files using `pd.read_parquet()`. Do not connect dashboards directly to Silver files, raw JSON, or any external API. The Gold layer is the single source of truth for the presentation layer.
- **Functional User Perspective:** For the storytelling dashboard, always ask: “Would my functional user understand this chart without any explanation?” If the answer is no, simplify the chart, improve its labels, or add a plain-language annotation.
- **Separation of Applications:** The governance and storytelling dashboards are two independent Plotly Dash applications running on different ports. Do not merge them into a single application — maintaining separation reflects the architectural decision made in the project design and keeps the audiences distinct.
- **Reproducibility is Non-Negotiable:** A project that cannot be reproduced from the repository is incomplete, regardless of dashboard quality. Test your setup instructions on a clean environment before submission.
- **Integration First:** Before investing time in visual polish, ensure the full pipeline runs correctly and both dashboards display accurate data. Integration correctness is a prerequisite for dashboard quality evaluation.

Good luck, and take pride in this final workshop — you are transforming months of data engineering work into a product that a real user can interact with and learn from. That transition from pipeline to insight is the essence of data analysis programming!